IMT-221 — Circuitos Electrónicos III

Hito 3: Guía Única E1 → E4 — Parte 4: Etapas E3 (Carga Inductiva) y E4 (Puente H Físico y Python)

Versión integrada 1.5 · 12 de noviembre de 2026 · Condicionado a firmas independientes G3 y G4

---

5. Etapa E3 — Inductancia Real y Recirculación de Corriente

5.1 Modelo Matemático y Trayectoria de Energía

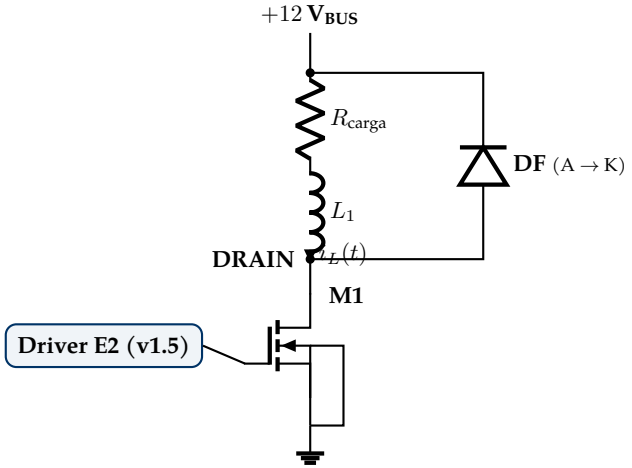
Para una bobina de inductancia L y resistencia serie R, durante el estado ON ideal ($V_{DS} \approx 0\,\text{V}$):

$$V_{\text{BUS}} = R\,i_L(t) + L\frac{di_L(t)}{dt}, \qquad \tau = \frac{L}{R}, \tag{1}$$

$$i_L(t) = \frac{V_{\text{BUS}}}{R}\left(1 - e^{-t/\tau}\right) + i_L(0)\,e^{-t/\tau}, \tag{2}$$

$$E_L = \int_0^{i_L} L\,i\,di = \frac{1}{2}L\,i_L^2 \quad [\text{J}]. \tag{3}$$

Al abrir el NMOS, la corriente no puede caer instantáneamente. Para la topología de bobina DC de una sola dirección, un diodo de recirculación `DF` se conecta en antiparalelo con la carga (**cátodo a $+V_{\text{BUS}}$, ánodo al nodo DRAIN**). No confundir el diodo intrínseco del NMOS con el diodo externo `DF`. Esta disposición no se traslada sin análisis al puente H de E4.

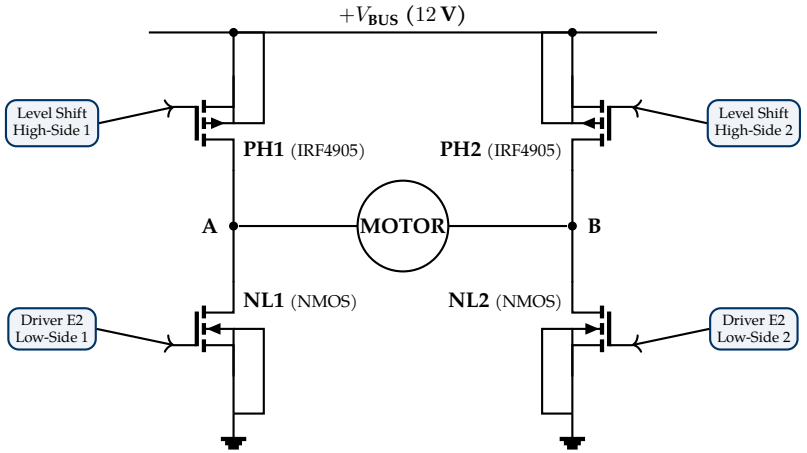


5.2 Pasos Obligatorios E3 (7 Pasos)

1. Registrar carga real: motor/bobina, tensión, corriente nominal, corriente de arranque/bloqueo, R, L y temperatura admisible.
2. Calcular $I_{\text{estacionaria}} \approx V_{\text{BUS}}/R$ o analizar la fuerza contraelectromotriz del motor declarando hipótesis.
3. Dimensionar `DF` (I_F, V_{RRM}, tiempo de recuperación y térmica) con ficha comercial.
4. Simular primero sin y luego con protección ($V_{DS}(t), i_L(t), i_{DF}(t), E_L$).
5. Montar con fuente apagada, comprobar polaridad de `DF`, driver E2 y presentar **G3**.
6. Con **G3** aprobado, ensayar a menor energía con pulsos supervisados; medir V_{GS}, V_{DS}, i_L, sobretensión y temperatura.
7. Comparar ON/OFF, estimación de τ y discutir por qué $V_{DS} > V_{\text{BUS}}$ durante recirculación.

6. Etapa E4 — Puente H Físico, Accionamiento Bidireccional y Pérdidas

6.1 Esquema Funcional (2 PMOS High-Side + 2 NMOS Low-Side)



Advertencia High-Side: Para PMOS con $V_S = +12\,\text{V}$, $V_G = 12\,\text{V} \implies V_{GS} = 0\,\text{V}$ (OFF, exige pull-up a $+V_{\text{BUS}}$); $V_G = 0\,\text{V} \implies V_{GS} = -12\,\text{V}$ (ON). GPIO 0–5 V directo al gate PMOS da $V_{GS} = -7\,\text{V}$ y **jamás asegura OFF**. Prohibido colocar pinza GND de osciloscopio en $+V_{\text{BUS}}$.

6.2 Tabla de Control de Estados y Dead-Time

Función	PH1	NL1	PH2	NL2	Observación
Reposo seguro	OFF	OFF	OFF	OFF	Sin accionamiento; verificar transitorios
Sentido $A \to B$	ON	OFF	OFF	ON	Par diagonal `PH1 + NL2`
Sentido $B \to A$	OFF	ON	ON	OFF	Par diagonal `PH2 + NL1`
Transición	OFF temp.	OFF temp.	OFF temp.	OFF temp.	**Dead-time / interbloqueo** entre estados
PROHIBIDO	**ON**	**ON**	cualq.	cualq.	**Corto de rama izquierda**
PROHIBIDO	cualq.	cualq.	**ON**	**ON**	**Corto de rama derecha**

6.3 Diseño Obligatorio Antes de G4 (8 Requisitos)

(1) Seleccionar 4 MOSFET reales y datasheets (`PH1/PH2`, `NL1/NL2`). (2) Diseñar por completo drivers high-side (level-shifter, pull-up a $+V_{\text{BUS}}$, arranque/fallo). (3) Dimensionar drivers low-side (E2 adaptado). (4) Implementar interbloqueo y *dead-time* superior a los tiempos reales de apagado/almacenamiento. (5) Comprobar motor, corriente de bloqueo, fusible y desacoplo. (6) Simular estados en LTspice (OFF, adelante, OFF, reversa y barrido de *dead-time*). (7) Prueba sin motor ni bus primero; referencia diferencial segura para V_{GS} high-side. (8) Ensayos físicos bajo **G4** sin inversión brusca ni shoot-through.

6.4 Estimación de Pérdidas / Análisis Térmico en Python

$$P_{\text{cond,med}} \approx \frac{1}{T}\int_0^T i_D(t)^2 R_{DS(\text{on})}(T_j, V_{GS})\,dt, \quad E_{\text{on/off}} = \int_{\text{evento}} v_{DS}(t)\,i_D(t)\,dt, \quad P_{\text{sw}} \approx f_{\text{sw}}(E_{\text{on}} + E_{\text{off}}), \quad P_{\text{gate}} \approx Q_g V_{\text{drive}} f_{\text{sw}}, \quad T_j \approx T_a + P_{\text{total}} R_{\theta JA}. \tag{4}$$

```
import pandas as pd
import numpy as np

df = pd.read_csv('evento_medido.csv')
t = df['t_s'].to_numpy(dtype=float)
v = df['vds_V'].to_numpy(dtype=float)
i = df['id_A'].to_numpy(dtype=float)
if len(t) < 2 or not np.all(np.diff(t) > 0):
    raise ValueError('Se requieren al menos dos tiempos crecientes')
p = v * i
E_J = np.trapz(p, t)
print(f'Energia integrada en intervalo medido: {E_J:.6e} J')
# Registrar intervalo exacto, calibracion, referencias, sincronizacion y error. Para P_sw se necesita E_on+E_off y frecuencia aplicable.
```